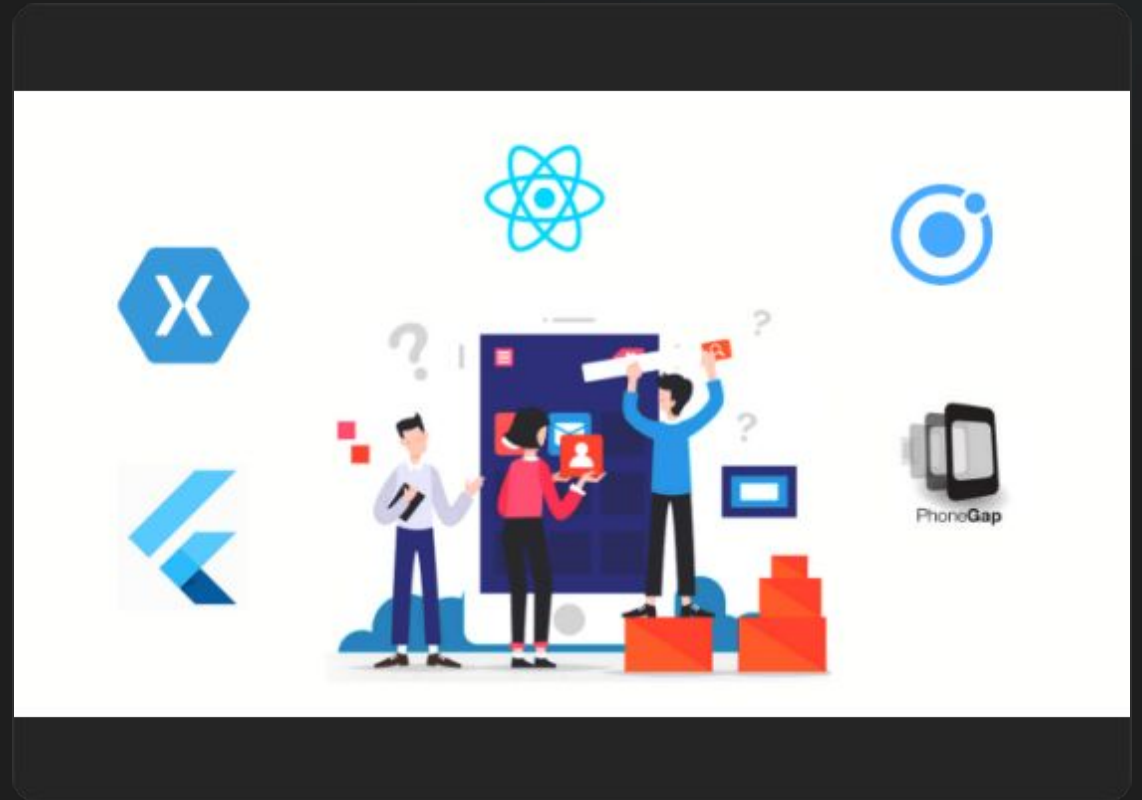


Construindo Apps Universais com React Native & Expo

Um guia prático para desenvolvedores web entrarem no mundo mobile com
uma única base de código (JavaScript/TypeScript).

O que é o **React Native**?

- ▶ Framework criado pelo Facebook (2015).
- ▶ Permite construir apps **nativos** para iOS e Android usando JavaScript e a biblioteca React.
- ▶ Não é uma "WebView" (site dentro de um app). O código JS comanda componentes nativos de verdade (UI).
- ▶ Oferece a experiência de desenvolvimento rápida da web com a performance e o visual do mobile nativo.
- ▶ Foca na reutilização de código de 80% a 90% entre as plataformas.

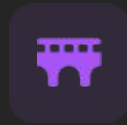


Como a mágica acontece sob o capô



1. A Lógica (JavaScript)

Você escreve seu código da mesma forma que na web, usando componentes React, hooks (useState, useEffect) e gerenciamento de estado.



2. A Ponte (The Bridge)

O coração do React Native. Ela traduz as instruções do seu código JavaScript para as linguagens nativas (Java/Kotlin no Android, Objective-C/Swift no iOS) em tempo real.



3. Componentes Nativos

Quando você usa uma tag , a ponte renderiza um ViewGroup no Android e um UIView no iOS. O usuário sente um app 100% nativo.

E onde entra o Expo nisso?

O React Native puro pode ser complexo de configurar. O Expo é o "canivete suíço" que simplifica tudo.

O Ecossistema Expo



- ▶ É um framework e uma plataforma para aplicativos universais React.
- ▶ **Sem código nativo no início:** Você não precisa instalar Android Studio ou Xcode para começar.
- ▶ **Expo Go:** Um app no seu celular que roda seu código instantaneamente durante o desenvolvimento via QR Code.
- ▶ **Expo SDK:** Fornece acesso fácil a APIs nativas (câmera, notificações, sensores, SQLite) sem precisar compilar código nativo manualmente.
- ▶ **EAS (Expo Application Services):** Serviços na nuvem para compilar, assinar e enviar o app para as lojas (Google Play/App Store).

1

Base de Código

Múltiplas Plataformas

Com o Expo e React Native, você escreve em JavaScript ou TypeScript uma única vez e o Expo gera aplicativos para **Android**, **iOS** e até mesmo para a **Web**.

Deixando o HTML para trás

No React Native, nós não usamos HTML. Nós usamos componentes primitivos fornecidos pelo framework que mapeiam diretamente para elementos da UI nativa.

- ▶ `View` se torna : O container base para layout com Flexbox.
- ▶ `Text` se tornam : Tudo que for texto DEVE estar dentro dessa tag.
- ▶ `Image` se torna : Para exibir imagens locais ou da rede.
- ▶ `TouchableOpacity` se torna `Pressable` ou `TouchableWithoutFeedback` : Para capturar toques.

Comparação

Na Web (React)
`Texto``Texto`

No Mobile (React Native)
`Texto`

Mão na Massa: O primeiro componente

Vamos criar um "Hello World" básico, seguindo o padrão inicial da documentação do Expo. Note como importamos os componentes do 'react-native'.

```
import { StyleSheet, Text, View } from 'react-native';

export default function App() {
  return (
    <View style={styles.container}>
      <Text style={styles.text}>Olá, Mundo Native!
      <Text>Este é meu primeiro app com Expo.
    </View>
  );
}

// A estilização não usa CSS puro, usa a API StyleSheet (JavaScript)
const styles = StyleSheet.create({
  container: {
    flex: 1, // Ocupa toda a tela
    backgroundColor: '#25292e',
    alignItems: 'center',
    justifyContent: 'center', // Centraliza no meio da tela (Flexbox)
  },
  text: {
    color: 'white',
    fontSize: 16,
  },
});
```


Neste exemplo omitimos o StyleSheet por brevidade. TouchableOpacity diminui a opacidade ao toque.

Adicionando Interatividade (Contador)

Como o React Native usa React, gerenciamos o estado da mesma forma com useState. Vamos adicionar um botão (TouchableOpacity) que altera a UI.

```
import React, { useState } from 'react';
import { StyleSheet, Text, View, TouchableOpacity } from 'react-native';

export default function Contador() {
  // Declaramos a variável 'count' e a função para atualizá-la
  const [count, setCount] = useState(0);

  return (
    style={styles.container}>
      style={styles.title}>Cliques: {count}

      style={styles.button}
      onPress={() => setCount(count + 1)}>
        style={styles.buttonText}>Aperte Aqui!

  );
};
```

Usando APIs do Dispositivo (Expo SDK)

O Expo facilita o uso de recursos de hardware sem tocar no código nativo. Exemplos práticos da documentação:



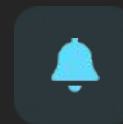
expo-image-picker

Permite abrir a galeria do usuário ou a câmera do sistema para selecionar fotos, lidando automaticamente com as permissões no iOS e Android.



expo-location

Lê as coordenadas GPS do dispositivo. Útil para mapas ou recursos geolocalizados, tudo feito através de chamadas assíncronas simples em JavaScript.



expo-notifications

Gerencia notificações push locais e remotas. O Expo fornece um serviço unificado que facilita o envio de mensagens para os aparelhos dos usuários.

Como começar seu projeto hoje

- ▶ **1. Instale o Node.js:** Pré-requisito básico.
- ▶ **2. Crie o projeto:** Use o comando oficial no seu terminal.
- ▶ **3. Inicie o servidor:** Rode `npx expo start` na pasta do projeto.
- ▶ **4. Baixe o Expo Go:** Disponível na App Store (iOS) e Google Play (Android).
- ▶ **5. Escaneie o QR Code:** Aponte a câmera do celular para o terminal e veja o app rodando!

No seu terminal:

```
$ npx create-expo-app@latest meu-app  
$ cd meu-app  
$ npx expo start
```



Escaneie o código gerado no terminal

Obrigado!

Você está pronto para começar a construir aplicativos nativos
incríveis com JavaScript.

Leia a documentação oficial

em:

docs.expo.dev

Image Sources



<https://agreemtech.com/wp-content/uploads/2021/07/Cross-Platform-App-Development.png>

Source: agreemtech.com



<https://static.expo.dev/static/home/2026/og-image.png>

Source: expo.dev